

TEORÍA (8 puntos)**Apellidos y nombre:****Pregunta 1. Explica en qué consiste una prueba de cobertura de condiciones y decisiones modificada****Pregunta 2. Relaciona los siguientes conceptos con su definición:**

- | | | | |
|-------|----|----|---|
| Dummy | 1● | ●a | Sustituto con una especificación del comportamiento que tiene que tener y de cómo va a ser invocados por la unidad testada. Son relevantes tanto las entradas como las salidas al colaborador, así como las invocaciones del objeto testado al colaborador. |
| Spy | 2● | ●b | Objetos (generalmente parámetros) que la unidad testada no va a usar en la prueba. |
| Stub | 3● | ●c | Objeto interpuesto entre un colaborador real y la unidad testada en el contexto de una prueba. |
| Mock | 4● | ●d | Sustituto que registra las salidas indirectas (salidas de la unidad testada al colaborador). La información más relevante es la que el sustituto recibe de la unidad testada. |
| Proxy | 5● | ●e | Objetos <i>ad hoc</i> para una prueba que proporcionan los resultados adecuados para esa prueba concreta. Solo es relevante la información que el sustituto proporciona a la unidad testada. |

Pregunta 3. ¿Qué es JUnit?**Pregunta 4. Da un par de ejemplos de riesgos del proceso de pruebas**

Pregunta 5. Define el término «cobertura»

Pregunta 6. Dentro del contexto de la plataforma *Cucumber* y el lenguaje mínimamente controlado *Gherkin*, relaciona las siguientes palabras reservadas con su utilización para describir en texto plano las distintas partes del comportamiento esperado del *software*:

When	1●	●a	Funcionalidad del objeto de pruebas
Feature	2●	●b	El estado esperado del sistema
Given	3●	●c	El cambio real sobre el sistema
Then	4●	●d	Caso de pruebas concreto.
Scenario	5●	●e	Precondiciones del sistema a probar

Pregunta 7. Enumera ordenadamente las actividades que se deben llevar a cabo para recolectar y analizar los riesgos de un producto dentro del plan de pruebas maestro.

Pregunta 8. Enumera dos técnicas que permitan estimar el número de horas total del proceso de pruebas, y una técnica que permita repartir un número de horas previamente conocido entre distintas actividades o componentes.

Pregunta 9. Explica brevemente cómo se puede estimar el rendimiento o beneficio de las pruebas desde un punto de vista económico.

Pregunta 10. ¿Qué se puede automatizar cuando se realizan pruebas de accesibilidad? Nombra alguna herramienta que conozcas.

EJERCICIO (2 puntos)**Prueba de caminos con profundidad**

Diseñad pruebas para el siguiente código utilizando la técnica de pruebas de caminos con nivel de profundidad igual a 2:

1. Obtened primero el grafo de flujo asociado, de acuerdo con la correspondencia entre instrucciones/condiciones y nodos que se indica en el propio código (0,25 puntos).
2. Hay dos condiciones relacionadas entre sí que harán que un par de aristas adyacentes cuya ejecución consecutiva es imposible. ~~Adicionalmente, la precondition del método hace que haya otro par de aristas adyacentes no puedan ejecutarse consecutivamente¹.~~ Identificad los dos pares de aristas adyacentes de ejecución consecutiva imposible (0,25 puntos).
3. Aplicad la técnica de pruebas de caminos con profundidad 2 para obtener el conjunto de caminos completos correspondiente. No consideréis como situaciones de prueba los dos pares de aristas de ejecución consecutiva imposible identificados en el punto anterior (0,75 puntos).
4. Finalmente, estableced casos de prueba para los caminos obtenidos, incluyendo entradas y salidas del método (0,75 puntos).

```

/**
 * Busca un dato determinado en un vector de enteros.
 * @param vector
 *         - el vector en el que se busca el dato «datoBuscado». Debe
 *         ser distinto de null y tener al menos una componente.
 * @param datoBuscado
 *         - el dato que se quiere buscar en el vector «vector».
 * @return Si en el vector «vector» hay un dato igual a «datoBuscado»,
 *         devuelve el índice de la componente en la que se encuentra.
 *         En caso contrario, devuelve -1;
 */
public static int buscar(int[] vector, int datoBuscado) {
    A int i = 0;

    while ( B i < vector.length - 1 && C vector[i] != datoBuscado) {
        D i++;
    }

    E if (vector[i] == datoBuscado) {
        F return i;
    } else {
        G return -1;
    }
    H }
}

```

1 En versiones preliminares del enunciado, las aristas ABE no se podían ejecutar de forma consecutiva. Tras un cambio en el código, la ejecución de estas aristas sí que es posible, pero no se eliminó la frase tachada del enunciado. Al corregir, se ha tenido en cuenta que el enunciado es erróneo.